```
for n in [1..59]
  water.stones.push new Swan
```

By writing the explanation first, you are forced to ponder how well the executable program matches the explanation. In other words, writing the explanation first can change how you think about your program.

Is it *really* literate programming?

Although Donald Knuth's literate programming inspired literate CoffeeScript, important aspects of Knuth's description are missing. That shouldn't discourage you from taking full advantage of it, though.

Literate CoffeeScript files aren't just for exploring poetry. A description of a program can serve as the starting point for creating a literate-style CoffeeScript program. Back to that Rot13 program then, how do you implement that with literate CoffeeScript?

Take the explanation of Rot13 and put it into a .litcoffee file. Then implement the program by putting *indented* program source throughout the explanation. The following listing shows a Rot13 program developed in exactly this way.

Listing 8.1. Literate CoffeeScript Rot13